

Authored by Aadarsh

# Master Object-Oriented Programming

A Complete Beginner's Guidebook — C++ Edition

**8 Weeks**

DURATION

**C++**

LANGUAGE

**Replit + Draw.io**

TOOLS

# Table of Contents

---

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>The Building Blocks of C++</b>                     | <b>3</b>  |
|          | Variables, Conditionals, Loops, Functions, Arrays     |           |
| <b>2</b> | <b>Introduction to OOP</b>                            | <b>5</b>  |
|          | Classes, Objects, Constructors, Destructors, Access   |           |
| <b>3</b> | <b>The Four Pillars of OOP</b>                        | <b>7</b>  |
|          | Encapsulation, Abstraction, Inheritance, Polymorphism |           |
| <b>4</b> | <b>Software Design &amp; UML</b>                      | <b>9</b>  |
|          | UML Diagrams, Final Project, Evaluation Criteria      |           |
| —        | <b>Common Mistakes &amp; Cheat Sheet</b>              | <b>11</b> |
|          | Debugging tips and quick syntax reference             |           |
| —        | <b>Worksheets &amp; Assessments</b>                   | <b>12</b> |
|          | Exercises and review questions                        |           |

## CHAPTER 1

# The Building Blocks of C++

Weeks 1–2 | Variables · Conditionals · Loops · Functions · Arrays

## 1

## 1.1 Variables and Data Types

A variable is a named storage location in memory. Every variable in C++ has a **type** that determines what kind of data it can hold and how much memory it occupies. Declaring a variable reserves that space and associates a name with it so you can read or modify its value throughout your program.

### Core Data Types

| Type                | Size    | Example Value                       | Common Use               |
|---------------------|---------|-------------------------------------|--------------------------|
| <code>int</code>    | 4 bytes | <code>int age = 21;</code>          | Whole numbers            |
| <code>double</code> | 8 bytes | <code>double gpa = 3.75;</code>     | Decimal / floating-point |
| <code>char</code>   | 1 byte  | <code>char grade = 'A';</code>      | Single character         |
| <code>bool</code>   | 1 byte  | <code>bool enrolled = true;</code>  | True / False logic       |
| <code>string</code> | varies  | <code>string name = "Alice";</code> | Text sequences           |

```
// Variable declarations
int studentID = 10423;
double gpa = 3.85;
string name = "Sarah";
bool graduated = false;

// Output to console
cout << "Student: " << name << ", GPA: " << gpa << endl;
```

## 1.2 Conditional Statements (if / else if / else)

Conditionals allow your program to make decisions at runtime. The CPU evaluates a boolean expression and branches into different code paths based on the result. This is the fundamental mechanism behind all decision-making logic in software.

```
int marks = 74;

if (marks >= 90) {
    cout << "Grade: A" << endl;
} else if (marks >= 75) {
    cout << "Grade: B" << endl;
} else if (marks >= 60) {
    cout << "Grade: C" << endl;
} else {
    cout << "Grade: F – Please see your instructor." << endl;
}
```

### Key Insight

Notice the use of `else if` to chain multiple conditions. Once one branch matches, the remaining branches are skipped entirely — order matters.

## 1.3 Loops — Repetition Without Repetition

A loop executes a block of code repeatedly until a condition is no longer true. C++ provides three loop constructs, each suited to different scenarios. Choosing the right one makes your intent clear to other developers reading your code.

### The for Loop — when you know the iteration count

Use a for loop when you know exactly how many times you need to iterate. It bundles the initialiser, condition, and increment into one line.

```
// Print a multiplication table for 7

for (int i = 1; i <= 10; i++) {
    cout << "7 x " << i << " = " << (7 * i) << endl;
}

// Iterating over an array

int scores[] = {88, 72, 95, 61, 84};

int total = 0;

for (int i = 0; i < 5; i++) {
    total += scores[i];
}
```

```
}  
  
cout << "Average: " << (total / 5.0) << endl;
```

## The while Loop — condition checked before each iteration

Use a while loop when the number of iterations is determined by runtime conditions, such as user input or file reading.

```
int attempts = 0;  
string password = "";  
  
while (password != "secure123" && attempts < 3) {  
    cout << "Enter password: ";  
    cin >> password;  
    attempts++;  
}  
  
if (password == "secure123") {  
    cout << "Access granted." << endl;  
} else {  
    cout << "Account locked." << endl;  
}
```

## The do-while Loop — executes at least once

```
// Menu that always displays at least once  
  
int choice;  
  
do {  
    cout << "1. View Records  2. Add Record  3. Exit" << endl;  
    cout << "Your choice: ";  
    cin >> choice;  
} while (choice != 3);
```

# 1.4 Functions

A function is a named, self-contained block of code that performs one specific task. Functions are the primary tool for code reuse and for keeping your programs modular, readable, and maintainable. The

golden rule: if you write the same logic twice, it belongs in a function.

```
// Function declaration (prototype)
double calculateGPA(int totalMarks, int subjects);

// Function definition
double calculateGPA(int totalMarks, int subjects) {
    if (subjects == 0) return 0.0;

    double average = (double)totalMarks / subjects;
    return average / 25.0; // scale to 4.0
}

// Calling the function
int main() {
    double gpa = calculateGPA(310, 4);
    cout << "GPA: " << gpa << endl;
    return 0;
}
```

## 1.5 Arrays and Vectors

When you need to store multiple values of the same type, arrays and vectors eliminate the need to declare dozens of individual variables. The key distinction: arrays have a **fixed size** set at compile time, while vectors are **dynamic** and can grow or shrink at runtime.

```
#include
#include

using namespace std;

// Fixed-size array
int grades[5] = {88, 92, 75, 61, 97};

// Dynamic vector – size can change
vector students;

students.push_back("Alice");
students.push_back("Bob");
students.push_back("Carol");
```

```
cout << "Enrolled: " << students.size() << " students" << endl;

// Range-based for loop (modern C++)
for (string& s : students) {
    cout << "    - " << s << endl;
}
```

## CHAPTER 2

# Introduction to OOP

Weeks 3–4 | Classes · Objects · Constructors · Destructors · Access Modifiers

## 2

## 2.1 The Paradigm Shift: From Procedures to Objects

Procedural programming treats a program as a sequence of instructions operating on data. Object-Oriented Programming (OOP) reorganises this model: data and the functions that operate on it are bundled together into **objects**. This mirrors how we naturally think about the real world — a car, a bank account, or a student record is a coherent entity with both properties and behaviours.

## 2.2 Classes and Objects

A **class** is a user-defined type — a blueprint specifying what data an object holds (attributes) and what operations it can perform (methods). An **object** is a concrete instance of that class, allocated in memory with its own copy of the attributes. A single class definition can produce thousands of independent objects.

```
class BankAccount {  
private:  
    string owner;  
    double balance;  
    string accountNumber;  
  
public:  
    // Constructor  
    BankAccount(string owner, string accNum, double initialDeposit) {  
        this->owner      = owner;  
        this->accountNumber = accNum;  
        this->balance     = initialDeposit;  
    }  
  
    void deposit(double amount) {  
        if (amount > 0) balance += amount;  
    }  
  
    bool withdraw(double amount) {
```



```
        if (amount > balance) return false;

        balance -= amount;

        return true;
    }

    double getBalance() const { return balance; }
    void printStatement() const {
        cout << "Account: " << accountNumber
              << " | Balance: $" << balance << endl;
    }
};

// Creating objects
BankAccount acc1("Alice", "NP-4421", 500.00);
BankAccount acc2("Bob", "NP-4422", 1200.00);

acc1.deposit(300.00);
acc1.withdraw(150.00);
acc1.printStatement(); // Account: NP-4421 | Balance: $650.00
```

## 2.3 Constructors and Destructors

A **constructor** is a special member function that runs automatically when an object is created. It has the same name as the class and no return type. Its job is to initialise the object into a valid state. C++ supports *constructor overloading* — you can define multiple constructors with different parameter lists.

A **destructor** runs automatically when an object goes out of scope or is explicitly deleted. It is responsible for releasing any resources the object acquired (memory, file handles, network connections). Its name is the class name prefixed with a tilde (~).

```
class FileLogger {
private:
    string filename;

public:
    // Default constructor
    FileLogger() {
```

```
        filename = "default.log";

        cout << "[Logger] Initialised with default log file." << endl;
    }

    // Parameterised constructor
    FileLogger(string fname) {
        filename = fname;

        cout << "[Logger] Opened: " << filename << endl;
    }

    // Destructor
    ~FileLogger() {
        cout << "[Logger] Closing file: " << filename << endl;

        // Resource cleanup happens here
    }

    void log(string message) {
        cout << "[" << filename << "]" " << message << endl;
    }
};
```

## 2.4 Access Modifiers — Public, Private, Protected

Access modifiers control visibility. This is not just a coding convention — the compiler enforces it, and violations produce compile-time errors.

| Modifier         | Accessible From                  | Typical Use                               |
|------------------|----------------------------------|-------------------------------------------|
| <b>private</b>   | Within the class only            | Attributes (data fields)                  |
| <b>public</b>    | Anywhere                         | Methods that form the interface           |
| <b>protected</b> | Class + subclasses (inheritance) | Base class internals shared with children |

### Best Practice

Always declare attributes as `private`. Expose them only through controlled `public` getter and setter methods. This gives you the power to validate data before it is stored — a cornerstone of robust software design.

## CHAPTER 3

# The Four Pillars of OOP

Weeks 5–6 | Encapsulation · Abstraction · Inheritance · Polymorphism

## 3

These four concepts are the theoretical foundation of object-oriented design. Every technical interview for a software engineering role will probe your understanding of at least one of them. Study the definitions, then — more importantly — study the code examples until the *why* becomes intuitive.

| Pillar        | Core Idea                               | Analogy                                                |
|---------------|-----------------------------------------|--------------------------------------------------------|
| Encapsulation | Bundle data + methods; hide internals   | <i>ATM — you use it, not service it</i>                |
| Abstraction   | Expose interface; hide implementation   | <i>Car pedal — no fuel injector knowledge needed</i>   |
| Inheritance   | Child class reuses parent class members | <i>Cat inherits from Animal</i>                        |
| Polymorphism  | One interface, multiple implementations | <i>Universal remote: one Play button, many devices</i> |

## 3.1 Encapsulation

Encapsulation means that an object's internal state is hidden from the outside world. External code can only interact with the object through its public interface — its methods. This prevents unintended interference, reduces coupling between components, and makes debugging dramatically easier because you always know where data changes can occur.

```
class Student {
private:
    string name;
    double gpa;           // hidden — cannot be set directly from outside

public:
    void setName(string n) { name = n; }
    string getName() const { return name; }

    void setGPA(double g) {
        if (g >= 0.0 && g <= 4.0) gpa = g;    // validation enforced here
        else cout << "Invalid GPA value." << endl;
    }
}
```

```

    }

    double getGPA() const { return gpa; }
};

Student s;

s.setName("Maria");

s.setGPA(5.5);    // rejected – invalid range

s.setGPA(3.8);    // accepted

// s.gpa = 5.5; <-- compile error: private member

```

## 3.2 Abstraction

Abstraction is the practice of hiding *how* something works and exposing only *what* it does. Abstract classes and interfaces define a contract — a set of methods that any concrete implementation must provide — without specifying the implementation details. In C++, this is achieved through abstract classes with **pure virtual functions**.

```

// Abstract base class – cannot be instantiated directly

class Shape {
public:
    virtual double area() const = 0;    // pure virtual
    virtual double perimeter() const = 0;
    virtual void describe() const {
        cout << "Shape with area=" << area() << endl;
    }
    virtual ~Shape() {}
};

class Circle : public Shape {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}

    double area() const override { return 3.14159 * radius * radius; }

    double perimeter() const override { return 2 * 3.14159 * radius; }
}

```

```
};

class Rectangle : public Shape {
private:
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) {}

    double area() const override { return width * height; }

    double perimeter() const override { return 2 * (width + height); }
};
```

### 3.3 Inheritance

Inheritance allows a new class (the *derived* or *child* class) to inherit attributes and methods from an existing class (the *base* or *parent* class). This promotes code reuse, establishes an "is-a" relationship between types, and forms the backbone of class hierarchies. The child class can extend or override parent behaviour.

```
class Animal {
protected:
    string name;
    int    age;
public:
    Animal(string n, int a) : name(n), age(a) {}

    virtual void speak() const { cout << name << " makes a sound." << endl; }

    void sleep() const { cout << name << " is sleeping." << endl; }
};

class Dog : public Animal {
private:
    string breed;
public:
    Dog(string n, int a, string b) : Animal(n, a), breed(b) {}

    void speak() const override { cout << name << " barks!" << endl; }

    void fetch() const { cout << name << " fetches the ball." << endl; }
```

```
};

class Cat : public Animal {
public:
    Cat(string n, int a) : Animal(n, a) {}

    void speak() const override { cout << name << " meows." << endl; }
};

Dog d("Rex", 3, "Labrador");
Cat c("Misty", 5);

d.speak(); // Rex barks!
c.sleep(); // Misty is sleeping. (inherited from Animal)
```

## 3.4 Polymorphism

Polymorphism (from Greek: "many forms") means that the same function call can produce different behaviour depending on the object it is called on. In C++, runtime polymorphism is achieved through **virtual functions** and base-class pointers or references. This is what allows you to write generic algorithms that work on entire families of related objects.

```
// Using base-class pointer for polymorphic behaviour
Animal* animals[3];

animals[0] = new Dog("Rex", 3, "Labrador");
animals[1] = new Cat("Misty", 5);
animals[2] = new Dog("Bruno", 2, "Beagle");

// Same call – different behaviour at runtime
for (int i = 0; i < 3; i++) {
    animals[i]->speak();
}

// Output:
// Rex barks!
// Misty meows.
// Bruno barks!

// Always delete dynamically allocated objects
```

```
for (int i = 0; i < 3; i++) delete animals[i];
```



# Software Design & Final Project

Weeks 7–8 | UML Diagrams · Architecture · Project Evaluation

# 4

## 4.1 UML Class Diagrams

UML (Unified Modeling Language) is a standardised visual notation for software design. Class diagrams let you plan the entire architecture of your application before writing a single line of code. In industry, a team would review and sign off on the UML design before development begins. Skipping this step is the leading cause of costly refactoring late in a project.

### Reading a UML Class Box

| Symbol | Meaning   | Example                            |
|--------|-----------|------------------------------------|
| +      | public    | + withdraw(amount : double) : bool |
| -      | private   | - balance : double                 |
| #      | protected | # name : string                    |
| *      | abstract  | * area() : double                  |

## 4.2 Final Project

You will design and implement a complete C++ application that demonstrates all four OOP pillars. Teams of one or two students. The project is submitted as source code plus a UML class diagram produced in Draw.io.

| Criterion           | Weight | What Assessors Look For                                         |
|---------------------|--------|-----------------------------------------------------------------|
| Creativity & Design | 40 %   | Novel problem domain; clear, purposeful class hierarchy         |
| Critical Thinking   | 40 %   | All four pillars evidenced; appropriate use of access modifiers |
| Element of Surprise | 10 %   | Unexpected feature; polished user experience                    |
| Presentation        | 10 %   | Code walkthrough; UML explained; Q&A; handled confidently       |

### Recommended Project Domains

|                                   |                                 |
|-----------------------------------|---------------------------------|
| ■ Library Management System       | ■ Hangman / Word-Guessing Game  |
| ■ Online Shopping Cart Simulation | ■ Socket-based Chat Application |
| ■ Bank Transaction System         | ■ Student Grade Management      |
| ■ Terminal-based Quiz Engine      | ■ Hospital Appointment System   |

## COMMON MISTAKES &amp; QUICK-REFERENCE

## Common Mistakes and How to Avoid Them

### 1. Missing Semicolons

C++ requires a `;` after every statement and after class/struct definitions. The compiler error will reference the line *after* the missing semicolon — always check one line above.

### 2. Confusing Assignment (=) and Equality (==)

`x = 5` assigns. `x == 5` compares. Writing `if (x = 5)` always evaluates to true and silently overwrites `x` — a notoriously difficult bug to find.

### 3. Public Attributes

Exposing attributes as `public` breaks Encapsulation. Any part of the codebase can modify the data with no validation. Always use `private` attributes with public getters/setters.

### 4. Memory Leaks

Every new must be paired with a `delete`. Forgetting to deallocate memory in destructors causes your program to consume increasing RAM until the OS kills it.

### 5. Skipping the UML Phase

Starting to code without a diagram almost always leads to a tightly coupled, unmaintainable class structure. Spend 30 minutes on a diagram — it saves hours of refactoring.

## QUICK-REFERENCE CHEAT SHEET

| Operation      | C++ Syntax                                                         |
|----------------|--------------------------------------------------------------------|
| Console output | <code>cout &lt;&lt; "text" &lt;&lt; variable &lt;&lt; endl;</code> |
| Console input  | <code>cin &gt;&gt; variable;</code>                                |
| Define a class | <code>class Name { private: ... public: ... };</code>              |
| Constructor    | <code>Name(params) { ... } // no return type</code>                |
| Destructor     | <code>~Name() { ... }</code>                                       |
| Getter method  | <code>int getValue() const { return value; }</code>                |
| Setter method  | <code>void setValue(int v) { value = v; }</code>                   |
| Inheritance    | <code>class Child : public Parent { };</code>                      |

|                         |                                                   |
|-------------------------|---------------------------------------------------|
| <b>Pure virtual fn</b>  | <code>virtual double area() const = 0;</code>     |
| <b>Override keyword</b> | <code>double area() const override { ... }</code> |
| <b>Dynamic array</b>    | <code>vector v; v.push_back(x);</code>            |
| <b>Range-based loop</b> | <code>for (Type item : collection) { ... }</code> |

## WORKSHEETS &amp; ASSESSMENTS

## Chapter 1 — Foundations Assessment

1. Explain the difference between a `float` and a `double` in C++. When would you choose one over the other?

*Model Answer:* `float` uses 4 bytes with ~7 significant digits; `double` uses 8 bytes with ~15. Use `double` in scientific or financial computing where precision is critical.

2. Write a C++ `for` loop that computes the sum of all even integers from 2 to 100 inclusive and prints the result.

*Model Answer:* `sum = 0; for(int i=2; i<=100; i+=2) sum+=i; cout << sum; // Expected: 2550`

3. What is the output of the following code? `int x=10; while(x>5) { cout<<x<<" "; x-=3; }`

*Model Answer:* 10 7 (x becomes 4 on the next iteration, failing the condition `x > 5`)

4. Declare a `vector<int>` named `primes`, populate it with the first five prime numbers, then print each element using a range-based `for` loop.

*Model Answer:* `vector primes = {2,3,5,7,11}; for(int p : primes) cout << p << " ";`

5. A function `isPrime(int n)` should return `true` if `n` is prime, `false` otherwise. Write its implementation.

*Model Answer:* `if(n<2) return false; for(int i=2;i<=sqrt(n);i++) if(n%i==0) return false; return true;`

## Chapter 2 — OOP Design Assessment

1. Distinguish between a class and an object. Provide an example domain and name at least two classes and two objects from it.

*Model Answer:* A class is a blueprint; an object is an instance. Domain: University. Classes: `Student`, `Course`. Objects: `student1` ("Alice", ID 1042), `course1` ("OOP 101").

2. Why should class attributes typically be declared `private`? What mechanism is used to give controlled external access?

*Model Answer:* Private attributes prevent direct external mutation. Controlled access is provided via public getter (accessor) and setter (mutator) methods, which can enforce validation.

3. Given a `Car` class with attributes `make`, `model`, and `speed`, write a parameterised constructor and a `accelerate(int delta)` method.

*Model Answer:* `Car(string mk,string md,int s):make(mk),model(md),speed(s){} void accelerate(int d){speed+=d; if(speed>200) speed=200;}`

#### 4. Explain what happens if you allocate an object with `new` but never call `delete`.

*Model Answer:* The object's destructor is never called, and its heap memory is never reclaimed. This is a memory leak — in long-running programs it can exhaust available RAM.

## Chapter 3 — Four Pillars Assessment

#### 1. Identify which OOP pillar each scenario illustrates: (a) A `Shape` base class with pure virtual `area()`. (b) A `Circle` that inherits from `Shape`. (c) Calling `s->area()` on a base pointer pointing to a `Circle`. (d) Private attributes accessed only via getters.

*Model Answer:* (a) Abstraction (b) Inheritance (c) Polymorphism (d) Encapsulation

#### 2. Write a C++ abstract class `Vehicle` with pure virtual method `fuelType()`, and two derived classes `ElectricCar` and `PetrolCar` that implement it.

*Model Answer:* `class Vehicle{public: virtual string fuelType() const=0; virtual ~Vehicle(){};};` `ElectricCar: fuelType(){return "Electric";}` `PetrolCar: fuelType(){return "Petrol";}`

#### 3. Explain, with code, how a base class pointer can call derived class methods at runtime. What keyword makes this possible?

*Model Answer:* The `virtual` keyword on the base class method enables dynamic dispatch. `Vehicle* v = new ElectricCar(); v->fuelType();` calls `ElectricCar::fuelType` at runtime.